

Task 1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
GNU nano 7.2                                     history.c *
int main() {
    char command[MAX_COMMAND];

    while (1) {
        printf("shell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            show_history();
            continue;
        }

        if (strlen(command) > 0)
            add_history(command);
    }

    return 0;
}

GNU nano 7.2                                     history.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

void add_history(char *command) {
    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void show_history() {
```

```

shreyas@Grigori:~$ mkdir -p ~/skill_session_11
shreyas@Grigori:~$ cd ~/skill_session_11
shreyas@Grigori:~/skill_session_11$ nano history.c
shreyas@Grigori:~/skill_session_11$ gcc history.c -o history
shreyas@Grigori:~/skill_session_11$ ./history
shell> ls
shell> pwd
shell> echo
shell> hello
shell> echo hello
shell> whoami
shell> /bin/ls
shell> ^C
shreyas@Grigori:~/skill_session_11$ nano history.c
shreyas@Grigori:~/skill_session_11$ ./history
shell> pwd
shell> ^C
shreyas@Grigori:~/skill_session_11$ nano pipeline.c
shreyas@Grigori:~/skill_session_11$ gcc pipeline.c -o pipeline
shreyas@Grigori:~/skill_session_11$ ./pipeline

```

Task 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```

GNU nano 7.2                                pipeline.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_LENGTH 100

typedef struct {
    char command[MAX_LENGTH];
} Command;

typedef struct {
    Command commands[MAX_COMMANDS];
    int count;
} Pipeline;

void add_command(Pipeline *pipeline, char *command) {
    if (pipeline->count < MAX_COMMANDS) {
        strcpy(pipeline->commands[pipeline->count].command, command);
        pipeline->count++;
    }
}

void display_pipeline(Pipeline *pipeline) {
    printf("Pipeline commands:\n");
}

```

```

    for (int i = 0; i < pipeline->count; i++) {
        printf("%d: %s\n",
               i + 1,
               pipeline->commands[i].command);
    }
}

int main() {

    Pipeline pipeline;
    pipeline.count = 0;

    char input[MAX_LENGTH];

    printf("Enter commands separated by |:\n");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, "|");

    while (token != NULL) {
        while (*token == ' ')
            token++;

```

```

shreyas@Grigori:~/skill_session_11$ nano history.c
shreyas@Grigori:~/skill_session_11$ ./history
shell> pwd
shell> ^C
shreyas@Grigori:~/skill_session_11$ nano pipeline.c
shreyas@Grigori:~/skill_session_11$ gcc pipeline.c -o pipeline
shreyas@Grigori:~/skill_session_11$ ./pipeline
Enter commands separated by |:
ls | grep .c | wc -l
Pipeline commands:
1: ls
2: grep .c
3: wc -l
shreyas@Grigori:~/skill_session_11$ cat file.txt | grep hello
cat: file.txt: No such file or directory
shreyas@Grigori:~/skill_session_11$ ls | sort | head
history
history.c
pipeline
pipeline.c
shreyas@Grigori:~/skill_session_11$ ./pipeline
Enter commands separated by |:
echo Hello | grep Hello | wc -l
Pipeline commands:
1: echo Hello
2: grep Hello
3: wc -l

```